UNIT 4
Hashing

Outline

- Definition
- Hash functions
- Overflow
- Collision
- Open Hashing
- Closed Hashing
- Rehashing Techniques

Lecture No.24

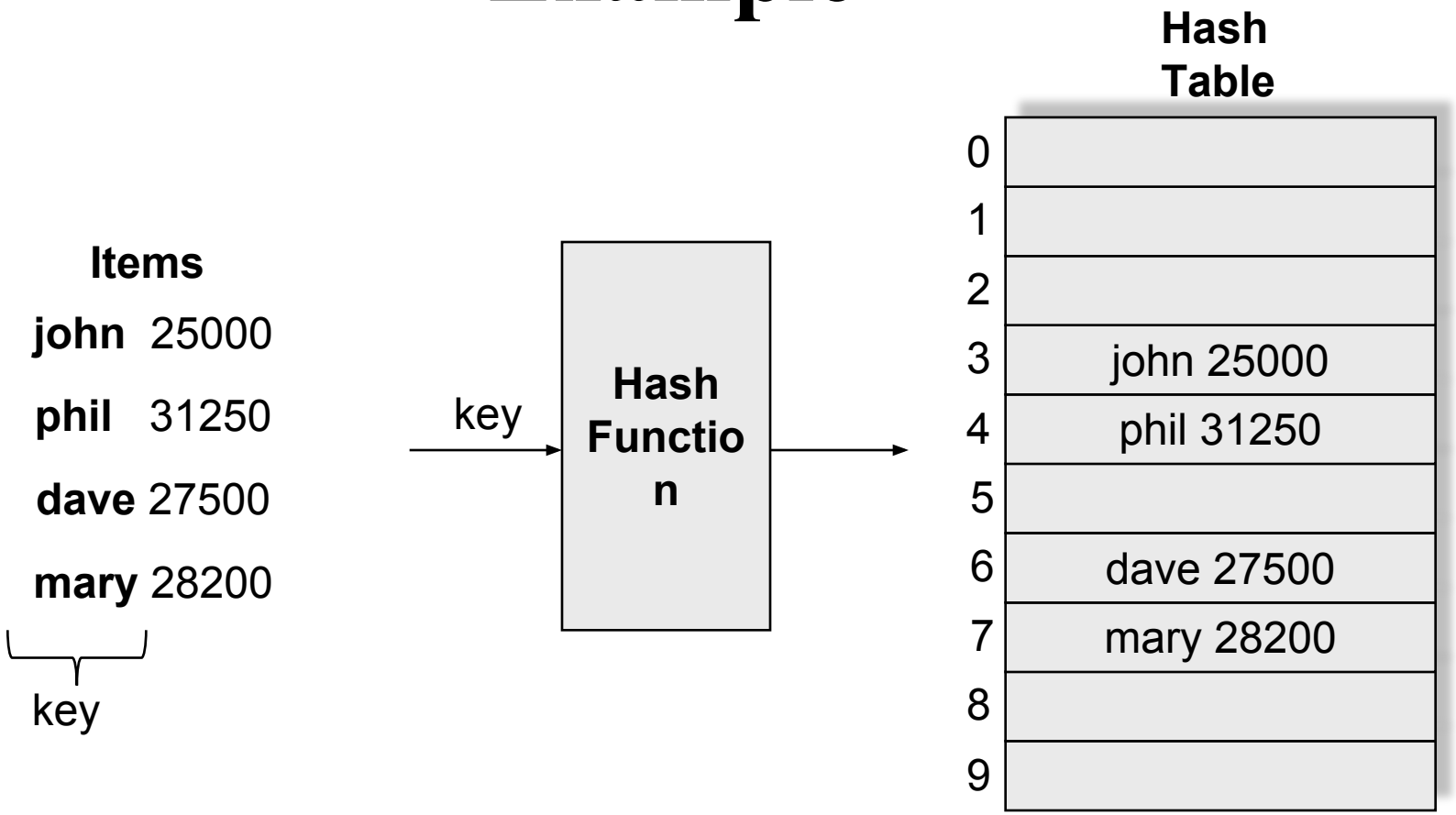
Hash Tables

- Hashing is one of the complex searching technique
- Consider a class of search techniques whose search time is dependents on the number of entries available in the table or the file which is determined by hash function
- Hashing is a technique used for performing insertions, deletions and finds in constant average time.
- The implementation of hash tables is called **hashing**.
- Data structure, however, is not efficient in operations that require any ordering information among the elements, such as findMin, findMax and printing the entire table in sorted order.

General Idea

- The ideal hash table structure is merely an array of some fixed size, containing the items.
- A stored item needs to have a data member, called ***key***, that will be used in computing the index value for the item.
  - Key could be an *integer*, a *string*, etc
  - e.g. a name or Id that is a part of a large employee structure
- The size of the array is *TableSize*.
- The items that are stored in the hash table are indexed by values from *0* to *TableSize – 1*.
- Each key is mapped into some number in the range 0 to *TableSize – 1*.
- The mapping is called a *hash function*.

Example


Items
john 25000
phil 31250
dave 27500
mary 28200
key
key
Hash Functio n
Hash Table
0
1
2
3
john 25000
4
phil 31250
5
6
dave 27500
7
mary 28200
8
9

Hash Function

- The hash function:
  - must be simple to compute.
  - must distribute the keys evenly among the cells.
- If we know which keys will occur in advance we can write *perfect* hash functions.

Hash Function

Problems:

- Keys may not be numeric.
- Number of possible keys is much larger than the space available in table.
- Different keys may map into same location
  - Hash function is not one-to-one => collision.
  - If there are too many collisions, the performance of the hash table will suffer dramatically.

Hash Functions

- If the input keys are integers then simply *Key* mod *TableSize* is a general strategy.
  - Unless key happens to have some undesirable properties. (e.g. all keys end in 0 and we use mod 10)
- If the keys are strings, hash function needs more care.
  - First convert it into a numeric value.

Some methods

1. Division Method

- Map a key k into one of the m slots by taking the remainder of k divided by m. That is,

$$h(k) = k \bmod m$$

- Example: $m = 31$ and $k = 78 \Rightarrow h(k) = 16$.
- Advantage: Fast, since requires just one division operation.
- Disadvantage: Have to avoid certain values of m.
  - Don't pick certain values, such as $m=2^p$
  - Or hash won't depend on all bits of k.
- Good choice for m:
  - Primes, not too close to power of 2 (or 10) are good.

2. MID Square Method

- multiply key by itself
- use some middle digits of the result as hash value.

When the key K is squared then the hash function H is defined by

H(K)=1

Example:

K= 453

K^2= 205209

H(K)= 52

3. Folding Method:

The key K is partitioned into number of parts, $K_{1 \ldots} K_r$. Then the parts are added together ignoring the last carry.

The hash function is defined as:

$$\mathbf{H(K)= K_1 + K_2 + \ldots K_r}$$

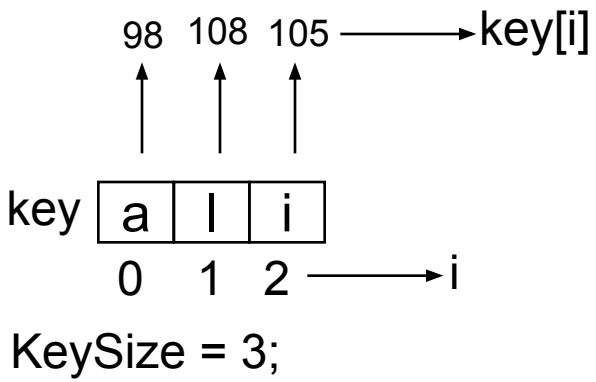
Example:

H(3205)=32+05=37

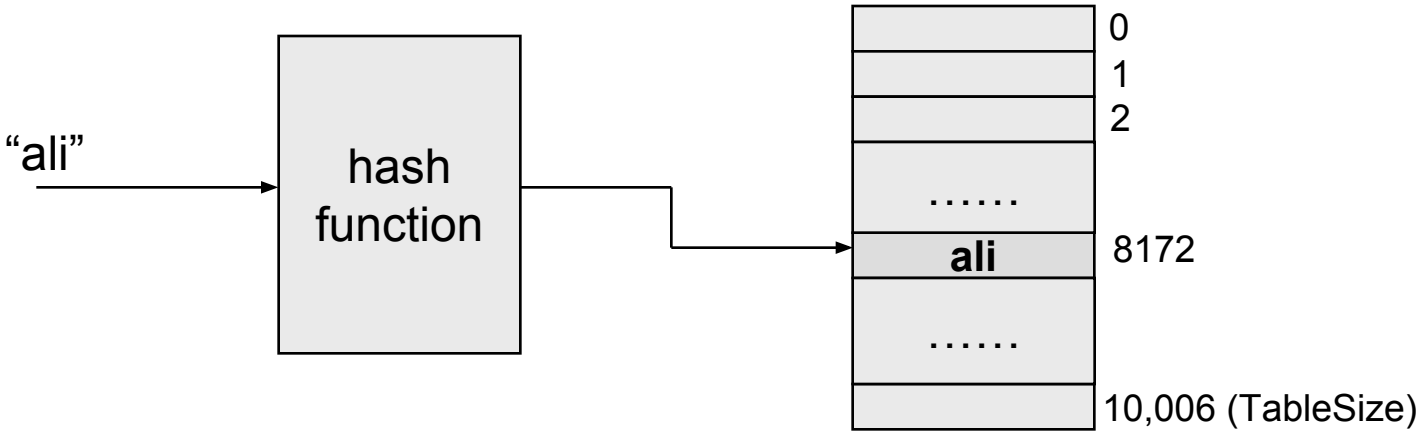
H(7148)=71+48=19

Observe that the leading digit 1 in H(7148) is ignored.

Hash function for strings:



hash("ali") = (105 * 1 + 108*37 + 98*37^2) % 10,007 = 8172



Overflow Handling

- An overflow occurs when the home bucket for a new pair (key, element) is full.

- We may handle overflows by:
  - Search the hash table in some systematic fashion for a bucket that is not full.
    - Linear probing (linear open addressing).
    - Quadratic probing.
    - Random probing.
  - Eliminate overflows by permitting each bucket to keep a list of all pairs for which it is the home bucket.
    - Array linear list.
    - Chain.

Collision Resolution

- If, when an element is inserted, it hashes to the same value as an already inserted element, then we have a collision and need to resolve it.
- There are several methods for dealing with this:
  - **Separate chaining**
  - **Open addressing**
    - Linear Probing
    - Quadratic Probing
    - Double Hashing

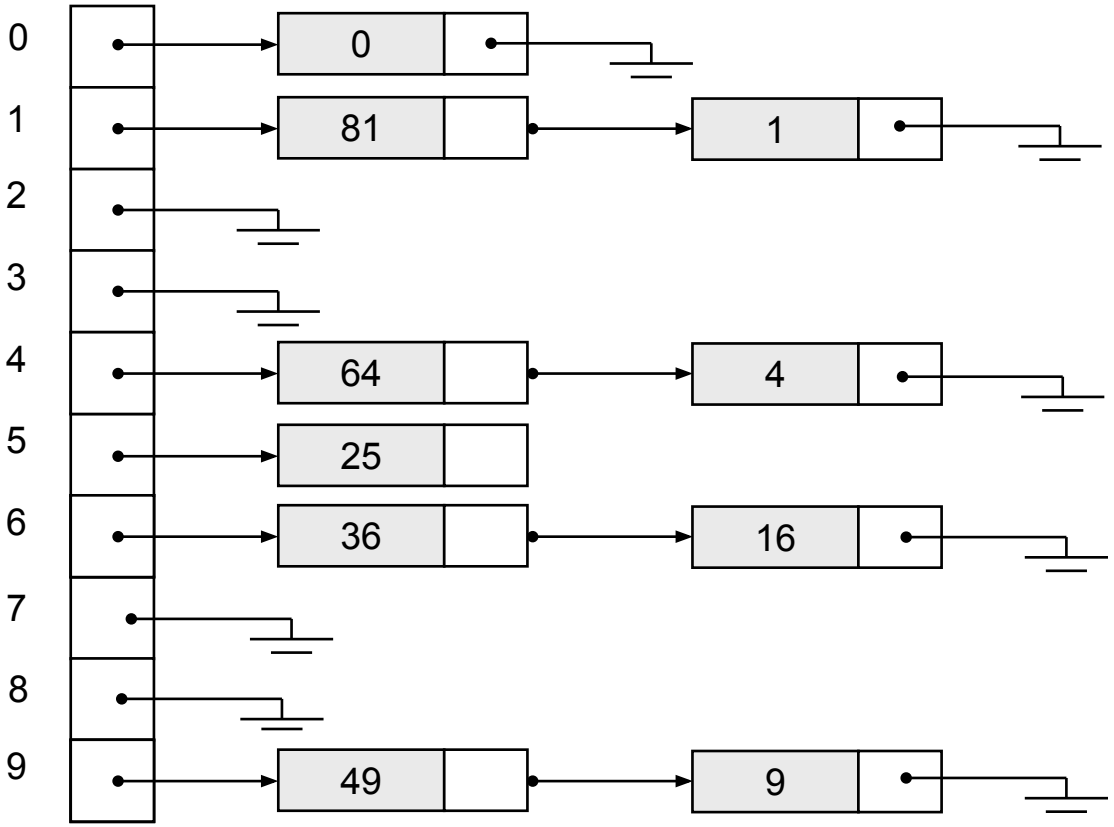
Separate Chaining

- The idea is to keep a list of all elements that hash to the same value.
  - The array elements are pointers to the first nodes of the lists.
  - A new item is inserted to the front of the list.
- Advantages:
  - Better space utilization for large items.
  - Simple collision handling: searching linked list.
  - Overflow: we can store more items than the hash table size.
  - Deletion is quick and easy: deletion from the linked list.

Example

Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81

hash(key) = key % 10.



Operations

- **Initialization**: all entries are set to NULL
- **Find**:
  - locate the cell using hash function.
  - sequential search on the linked list in that cell.
- **Insertion**:
  - Locate the cell using hash function.
  - (If the item does not exist) insert it as the first item in the list.
- **Deletion**:
  - Locate the cell using hash function.
  - Delete the item from the linked list.

Analysis of Separate Chaining

- Collisions are very likely.
  - How likely and what is the average length of lists?
- Load factor λ definition:
  - Ratio of number of elements (N) in a hash table to the hash *TableSize*.
    - i.e. $\lambda = N/TableSize$
  - The average length of a list is also λ.
  - For chaining λ is not bound by 1; it can be > 1.

Cost of searching

- **Cost** = Constant time to evaluate the hash function + time to traverse the list.
- **Unsuccessful search**:
  - We have to traverse the entire list, so we need to compare l nodes on the average.
- **Successful search**:
  - List contains the one node that stores the searched item + 0 or more other nodes.
  - Expected # of other nodes = x = (N-1)/M which is essentially l, since M is presumed large.
  - On the average, we need to check *half* of the *other nodes* while searching for a certain element
  - Thus average search cost = 1 + l/2

Hashing: Open Addressing

Collision Resolution with Open Addressing

- Separate chaining has the disadvantage of using linked lists.
  - Requires the implementation of a second data structure.
- In an open addressing hashing system, all the data go inside the table.
  - Thus, a bigger table is needed.
    - Generally the load factor should be below 0.5.
  - If a collision occurs, alternative cells are tried until an empty cell is found.

Open Addressing

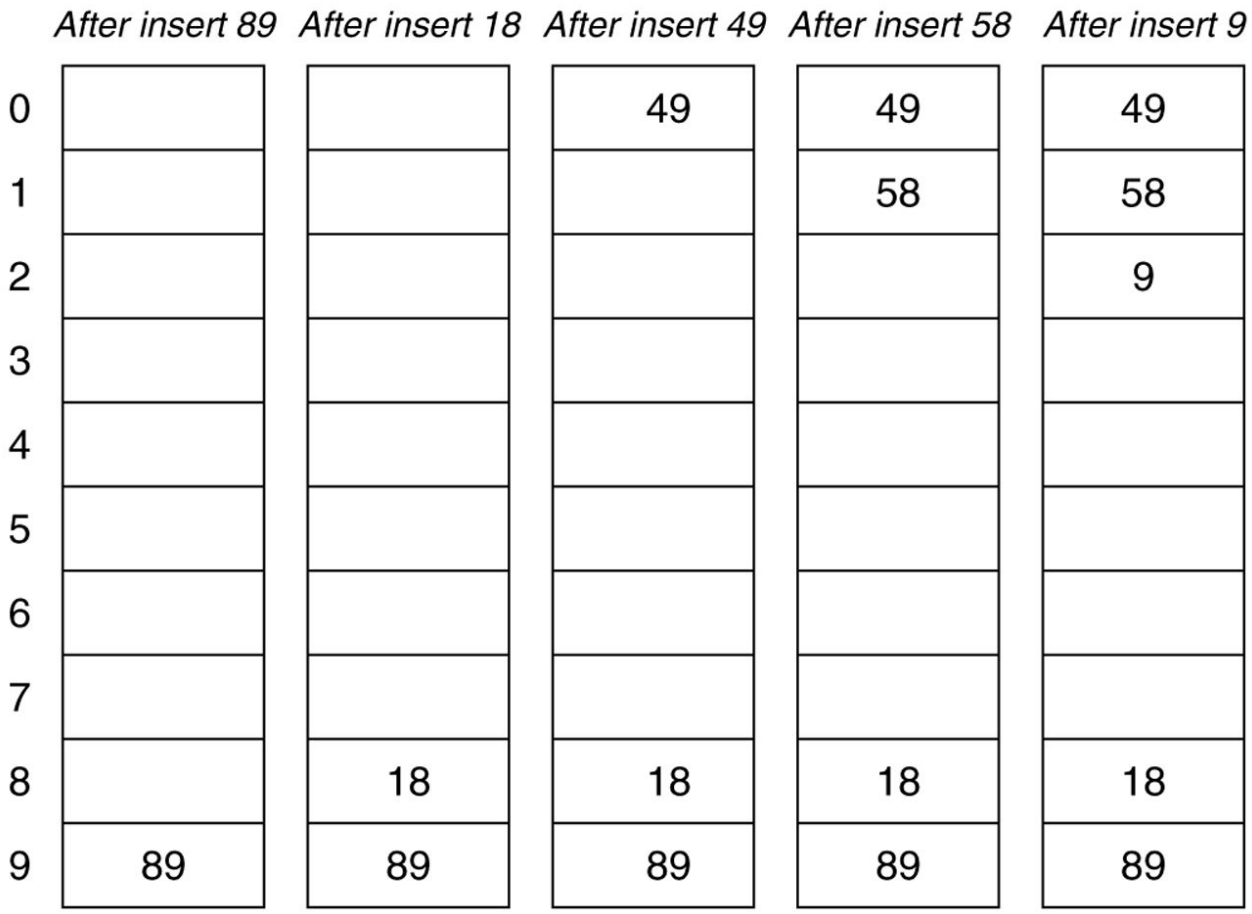
- More formally:
  - Cells $h_0(x)$, $h_1(x)$, $h_2(x)$, ...are tried in succession where $h_i(x) = (hash(x) + f(i))$ mod *TableSize,* with $f(0) = 0$.
  - The function *f* is the collision resolution strategy.
- There are three common collision resolution strategies:
  - Linear Probing
  - Quadratic probing
  - Double hashing

Linear Probing

- In linear probing, collisions are resolved by sequentially scanning an array (with wraparound) until an empty cell is found.
  - i.e. *f* is a linear function of *i*, typically *f(i)* = *i*.
- Example:
  - Insert items with keys: 89, 18, 49, 58, 9 into an empty hash table.
  - Table size is 10.
  - Hash function is hash(x) = x mod 10.
    - *f(i)* = *i*;

Figure
Linear probing hash table after each insertion

```
hash ( 89, 10 ) = 9
hash ( 18, 10 ) = 8
hash ( 49, 10 ) = 9
hash ( 58, 10 ) = 8
hash (  9, 10 ) = 9
```



Find and Delete

- The find algorithm follows the same probe sequence as the insert algorithm.
  - A find for 58 would involve 4 probes.
  - A find for 19 would involve 5 probes.
- We must use *lazy deletion* (i.e. marking items as deleted)
  - Standard deletion (i.e. physically removing the item) cannot be performed.
  - e.g. remove 89 from hash table.

Analysis of insertion

- The average number of cells that are examined in an insertion using linear probing is roughly

$$(1 + 1/(1 - \lambda)^2) / 2$$

  - Proof is beyond the scope of text book.
- For a half full table we obtain 2.5 as the average number of cells examined during an insertion.
- Primary clustering is a problem at high load factors. For half empty tables the effect is not disastrous.

Analysis of Find

- An unsuccessful search costs the same as insertion.
- The cost of a successful search of X is equal to the cost of inserting X at the time X was inserted.
- For $\lambda = 0.5$ the average cost of insertion is 2.5. The average cost of finding the newly inserted item will be 2.5 no matter how many insertions follow.
- Thus the average cost of a successful search is an average of the insertion costs over all smaller load factors.

Average cost of `find`

- The average number of cells that are examined in an unsuccessful search using linear probing is roughly $(1 + 1/(1 - \lambda)^2) / 2$.
- The average number of cells that are examined in a successful search is approximately $(1 + 1/(1 - \lambda)) / 2$.
  - Derived from:

$$\frac{1}{\lambda}\int_{x=0}^{\lambda}\frac{1}{2}\left(1+\frac{1}{(1-x)^2}\right)dx$$

Linear Probing – Analysis -- Example

- What is the average number of probes for a successful search and an unsuccessful search for this hash table?
  - Hash Function: h(x) = x mod 11

Successful Search:

- 20: 9 -- 30: 8 -- 2 : 2 -- 13: 2, 3 -- 25: 3,4
- 24: 2,3,4,5 -- 10: 10 -- 9: 9,10, 0

Avg. Probe for SS = (1+1+1+2+2+4+1+3)/8=15/8

Unsuccessful Search:

- We assume that the hash function uniformly distributes the keys.
- 0: 0,1 -- 1: 1 -- 2: 2,3,4,5,6 -- 3: 3,4,5,6
- 4: 4,5,6 -- 5: 5,6 -- 6: 6 -- 7: 7 -- 8: 8,9,10,0,1
- 9: 9,10,0,1 -- 10: 10,0,1

Avg. Probe for US =
(2+1+5+4+3+2+1+1+5+4+3)/11=31/11

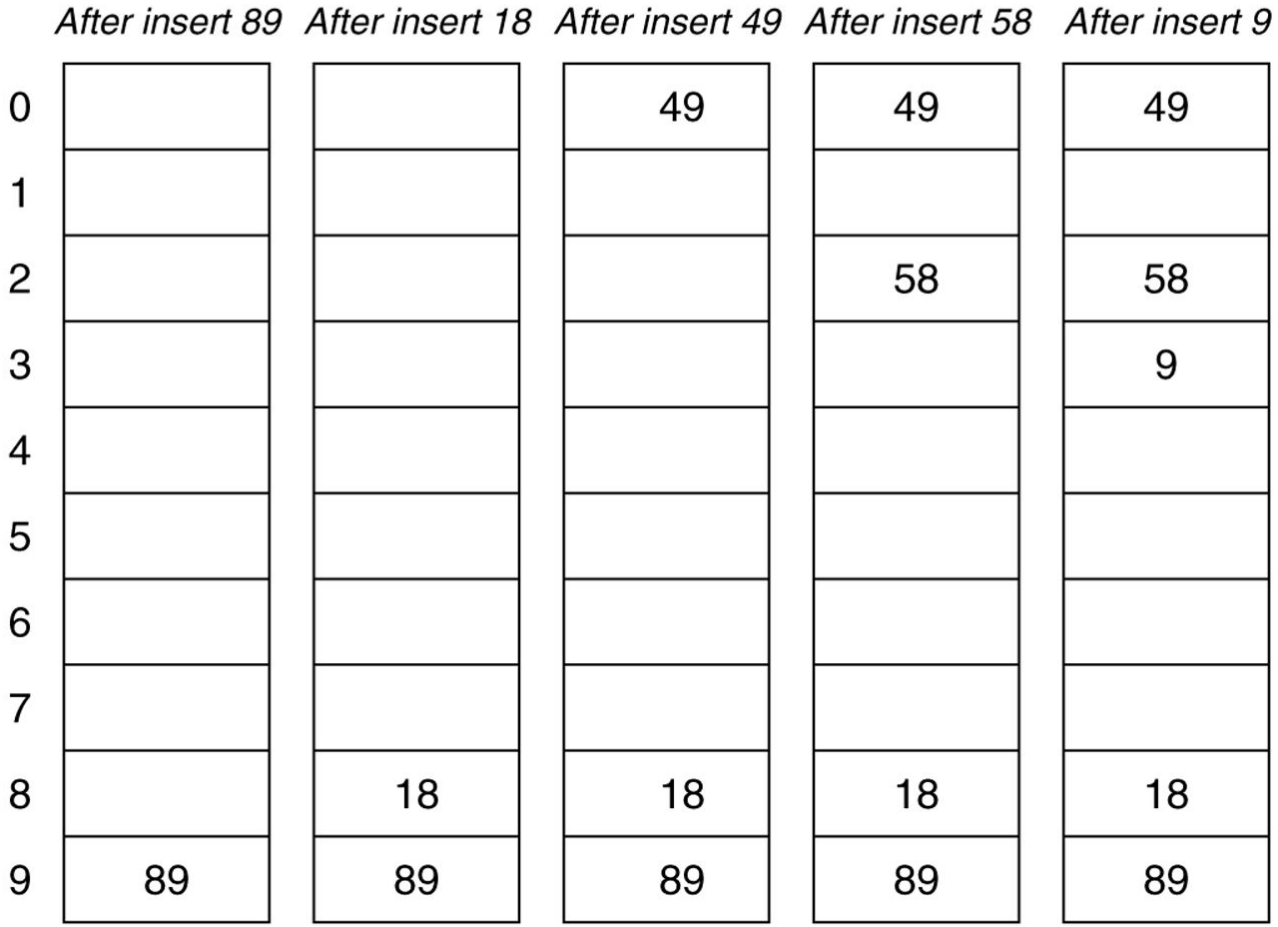
0	9
1	
2	2
3	13
4	25
5	24
6	
7	
8	30
9	20
10	10

Quadratic Probing

- Quadratic Probing eliminates primary clustering problem of linear probing.
- Collision function is quadratic.
  - The popular choice is $f(i) = i^2$.
- If the hash function evaluates to h and a search in cell h is inconclusive, we try cells $h + 1^2$, $h+2^2$, … $h + i^2$.
  - i.e. It examines cells 1,4,9 and so on away from the original probe.
- Remember that subsequent probe points are a quadratic number of positions from the *original probe point*.

Figure
A quadratic probing hash table after each insertion (note that the table size was poorly chosen because it is not a prime number).

```
hash ( 89, 10 ) = 9
hash ( 18, 10 ) = 8
hash ( 49, 10 ) = 9
hash ( 58, 10 ) = 8
hash (  9, 10 ) = 9
```



Quadratic Probing

- Problem:
  - We may not be sure that we will probe all locations in the table (i.e. there is no guarantee to find an empty cell if table is more than half full.)
  - If the hash table size is not prime this problem will be much severe.
- However, there is a theorem stating that:
  - If the table size is *prime* and load factor is not larger than 0.5, all probes will be to different locations and an item can always be inserted.

Some considerations

- How efficient is calculating the quadratic probes?
  - Linear probing is easily implemented. Quadratic probing appears to require * and % operations.
  - However by the use of the following trick, this is overcome:
    - $H_i = H_{i-1} + 2i - 1 \pmod{M}$

Some Considerations

- What happens if load factor gets too high?
  - Dynamically expand the table as soon as the load factor reaches 0.5, which is called *rehashing*.
  - Always double to a prime number.
  - When expanding the hash table, reinsert the new table by using the new hash function.

Analysis of Quadratic Probing

- Quadratic probing has not yet been mathematically analyzed.
- Although quadratic probing eliminates primary clustering, elements that hash to the same location will probe the same alternative cells. This is know as *secondary clustering*.
- Techniques that eliminate secondary clustering are available.
  - the most popular is *double hashing*.

Double Hashing

- A second hash function is used to drive the collision resolution.
  - $f(i) = i * hash_2(x)$
- We apply a second hash function to x and probe at a distance $hash_2(x)$, $2*hash_2(x)$, ... and so on.
- The function $hash_2(x)$ must never evaluate to zero.
  - e.g. Let $hash_2(x) = x \bmod 9$ and try to insert 99 in the previous example.
- A function such as $hash_2(x) = R - (x \bmod R)$ with R a prime smaller than TableSize will work well.
  - e.g. try R = 7 for the previous example.(7 - x mode 7)

Hashing Applications

- Compilers use hash tables to implement the *symbol table* (a data structure to keep track of declared variables).
- Game programs use hash tables to keep track of positions it has encountered (*transposition table*)
- Online spelling checkers.

Open hashing

- Two classes of collision resolution techniques:
  - *Open hashing* … also known as separate chaining
  - *Closed hashing* … also known as open addressing
- With open hashing, collisions lead to records being stored outside the hashing table
- With closed hashing, collisions lead to records being stored in the table at a position different from that calculated by the hashing function.
- **Open Hashing Implementation**
- A simple design for open hashing defines each slot in the table to be the head of a linked list. All records hashed to that slot are then placed in this linked list.
- Now we need only decide how records are ordered in each list -- the most popular techniques are by insertion time, or key value order, or frequency of access time.

When to use open hashing:

- It is most appropriate when the hash table is to be kept in main memory; storing such a table on disk would be very inefficient.
- Note: open hashing is based on the same idea as the *bin sort* (which we will see later).

Closed Hashing

- Closed hashing stores all records directly in the hash table.
- Each record has a *home position* defined by h(k), where k is the record's key.
- If a record R is to be inserted and another record already occupies R's home then R will be stored at some other slot.
- The collision resolution policy determines which slot.
- The same policy must also be followed when searching the database.
- A simple example:
- if a collision occurs just move onto the next position in the table. If the end of the table is reached then just loop around to the front.

Closed hashing continued ...

- Bucket Hashing … group hash table slots into buckets
- Hash table: array of M slots divided into B buckets, (M/B slots per bucket)
- Hash function maps to buckets (1st slot free)
- **Overflow bucket** --- when a bucket is full then record is store here.
- All buckets share the same **overflow**
- Goal: minimise use of overflow!
- Searching:
  - hash the key to determine the bucket
  - if key not found in bucket and bucket is not full then search is complete
  - if key not found and bucket is full then check the overflow bucket
- Note: searching the overflow can be expensive if there are many records contained within

Rehashing

- Increases the size of the hash table when load factor becomes “too high” (defined by a cutoff)
  - Anticipating that prob(collisions) would become higher
- Typically expand the table to twice its size (but still prime)
- Need to reinsert all existing elements into new hash table

Rehashing Example

h(x) = x mod 7
λ = 0.57

0	6
1	15
2	
3	24
4	
5	
6	13

Insert 23

λ = 0.71

0	6
1	15
2	23
3	24
4	
5	
6	13

Rehashing

h(x) = x mod 17
λ = 0.29

0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	

Rehashing Analysis

- Rehashing takes time to do N insertions
- Therefore should do it infrequently
- Specifically
  - Must have been N/2 insertions since last rehash
  - Amortizing the O(N) cost over the N/2 prior insertions yields only constant additional time per insertion

Rehashing Implementation

- When to rehash
  - When load factor reaches some threshold (e.g,. $\lambda \geq 0.5$), OR
  - When an insertion fails

- Applies across collision handling schemes